



JDBC Programming with Java

Java Database Connectivity (JDBC) is a core API that enables Java applications to interact with databases since its introduction in JDK 1.1 (1997). This powerful interface allows developers to write database-agnostic code that works seamlessly with any SQL-compliant database system.

JDBC Architecture



Java Application

Your application code makes JDBC API calls to connect to and query databases



JDBC API

Standard interfaces and classes in the java.sql package



JDBC Driver API

Database-specific implementations that handle the actual communication



Database

Target database system (MySQL, Oracle, PostgreSQL, etc.)

The architecture supports both two-tier (direct) and three-tier (middle-tier) connectivity models, allowing multiple concurrent drivers to work with different database systems simultaneously.

Key JDBC Components



1

DriverManager

Loads database drivers and creates connections to databases

2

Connection

Represents a session with a specific database

3

Statement Types

Execute SQL queries and updates (Statement, PreparedStatement, CallableStatement)

4

ResultSet

Contains data retrieved from database queries

5

SQLException

Handles errors and exceptions from database operations

Establishing a Database Connection

Connection Process

1. Load the database driver (automatic in JDBC 4.0+)
2. Establish connection via DriverManager or DataSource
3. Configure connection properties as needed
4. Handle connection errors appropriately

 Modern applications should use connection pooling via DataSource for improved performance and resource management.

```
// Example connection code
String url = "jdbc:mysql://localhost:3306/mydb";
String username = "user";
String password = "pass";

try (Connection conn =
    DriverManager.getConnection(url, username, password)) {
    // Connection is established
    System.out.println("Connected to database!");
    // Perform database operations...
} catch (SQLException e) {
    e.printStackTrace();
}
```

Executing SQL Statements

Statement

Used for executing static SQL statements. Each query is compiled and executed at runtime.

```
Statement stmt =  
conn.createStatement();  
ResultSet rs =  
stmt.executeQuery("SELECT *  
FROM employees");
```

PreparedStatement

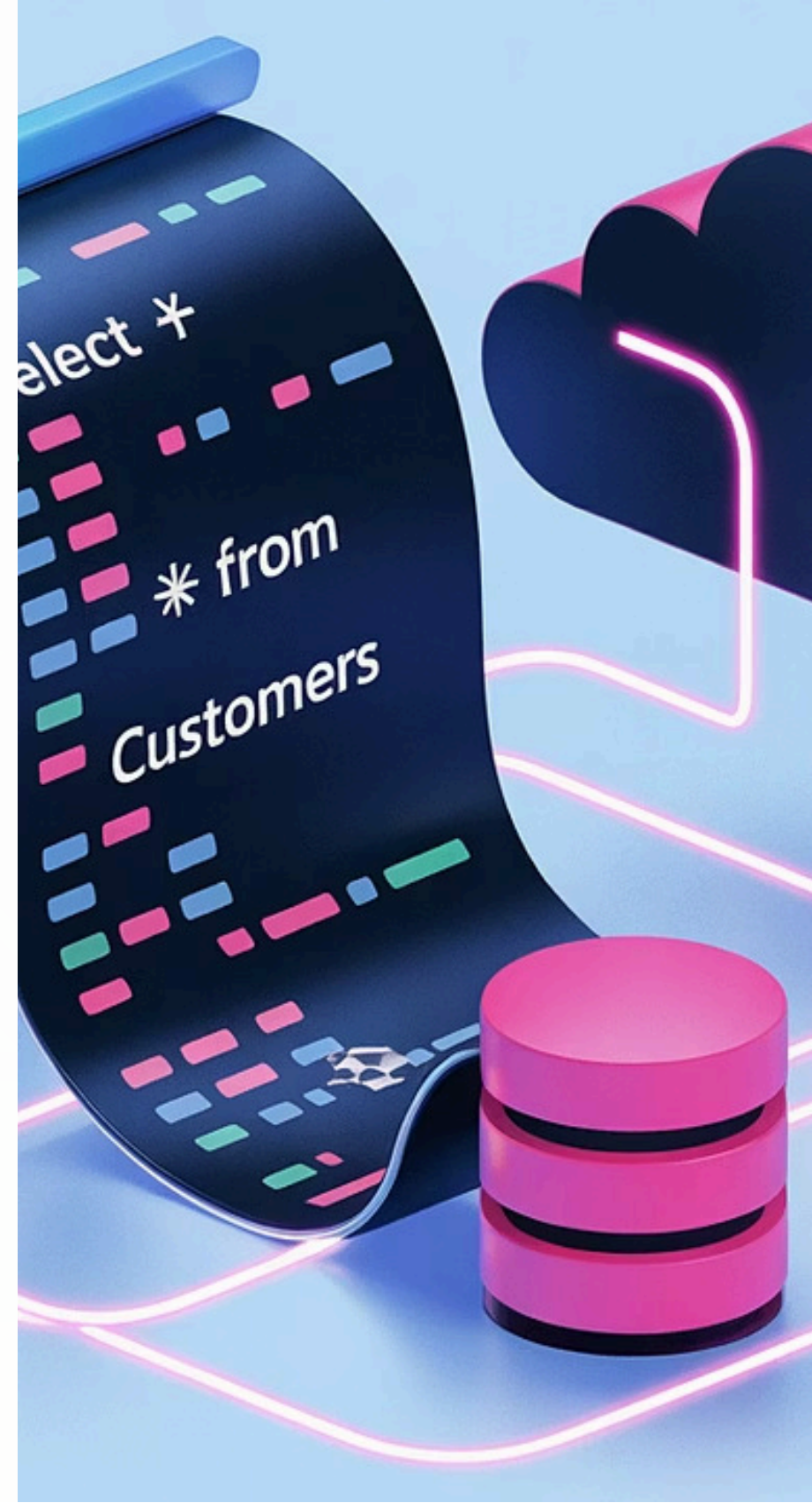
Precompiled statements that improve performance for repeated execution. Prevents SQL injection.

```
PreparedStatement pstmt =  
conn.prepareStatement(  
    "SELECT * FROM employees  
WHERE dept = ?");  
pstmt.setString(1,  
    "Engineering");  
ResultSet rs =  
pstmt.executeQuery();
```

CallableStatement

Used to call stored procedures in the database.

```
CallableStatement cstmt = conn.prepareCall(  
    "{call get_employee_by_id(?)}");  
cstmt.setInt(1, 101);  
ResultSet rs = cstmt.executeQuery();
```



Retrieving and Processing Results



Working with ResultSet

```
// Execute a query
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(
    "SELECT id, name, salary FROM employees");

// Process the results
while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    double salary = rs.getDouble("salary");

    System.out.println(id + ": " + name +
        " (£" + salary + ")");
}

// Close resources
rs.close();
stmt.close();
```

Use appropriate getXXX() methods based on column data types. Always close resources to prevent memory leaks.

Best Practices & Error Handling

Resource Management

Use try-with-resources to automatically close JDBC resources:

```
try (
    Connection conn =
dataSource.getConnection();
    PreparedStatement pstmt =
conn.prepareStatement(sql);
    ResultSet rs = pstmt.executeQuery()
){
    // Process results
}
```

SQL Injection Prevention

Always use PreparedStatement with parameter binding instead of string concatenation:

```
// GOOD:
pstmt.setString(1, userInput);

// BAD:
stmt.execute("SELECT * FROM users
            WHERE name='" + userInput +
            "'");
```

Connection Pooling

Implement connection pooling to improve performance and scalability:

- HikariCP (recommended)
- Apache DBCP
- C3P0
- Tomcat JDBC Pool

Implement proper exception handling to catch and log SQLExceptions with specific error codes and messages to aid in troubleshooting database issues.

Summary & Future Trends

Key Takeaways

- JDBC provides standardised database access for Java applications
- Core components include Connection, Statement, and ResultSet
- PreparedStatement offers performance and security benefits
- Resource management is critical for robust applications
- JDBC remains essential for Java developers working with databases

Next Steps

- Explore JDBC 4.3 enhancements in Java SE 9+
- Learn about integrations with modern frameworks
- Implement connection pooling in your applications



While ORM frameworks like Hibernate and JPA have gained popularity, understanding JDBC fundamentals remains crucial as these frameworks are built on JDBC's foundation.

Modern applications increasingly combine JDBC with reactive programming models and NoSQL databases to handle diverse data storage requirements.